
crobat Documentation

Release 1.0.0

Ivan E. Perez

May 03, 2026

CONTENTS:

1	Getting Started	3
1.1	Prerequisites	3
1.2	Installation	3
1.3	Using the CLI	4
1.4	Using the Python API	4
1.5	Handling snapshot timeouts	5
2	Output Format	7
2.1	Output files	7
2.2	Order book snapshots	7
2.3	Signed order book snapshot	8
2.4	Event recordings	8
3	API Reference	9
3.1	crobat.recorder	9
3.2	crobat.orderbook	11
3.3	crobat.orderbook_helpers	17
3.4	crobat.filesave	18
3.5	crobat.config	21
4	References	23
4.1	Order book dynamics and queue-reactive models	23
4.2	Stochastic order book models	23
4.3	Signed order flow and price impact	23
4.4	Spoofing and manipulation	23
4.5	Cryptocurrency order flow analysis	24
4.6	Related work	24
5	Indices and tables	25
	Python Module Index	27
	Index	29

crobat is a Python library for recording live Level 2 order book data from the Coinbase Advanced Trade WebSocket feed. It captures limit order insertions, cancellations, and market orders in real time and saves them as structured time series files.

GETTING STARTED

1.1 Prerequisites

- Python 3.10 or higher
- A Coinbase Advanced Trade account with API credentials (`cdp_api_key.json` downloaded from the Coinbase Developer Portal)

Dependencies are declared in `setup.py`. Key packages:

- `coinbase-advanced-py==1.8.2`
- `numpy`
- `pandas`

1.2 Installation

Clone the repository:

```
git clone https://github.com/orderbooktools/crobat.git
cd crobat
```

Place your `cdp_api_key.json` in the project root. The `config.ini` file controls default recording settings:

```
[recording]
currency_pair      = XRP-USD
position_range     = 5
recording_duration = 10
sides              = bid,ask,signed
filetype           = csv
```

Install the package in editable mode so the `crobat` CLI command is available system-wide:

```
pip install -e .
```

1.3 Using the CLI

Run from the project root:

```
# Use defaults from config.ini
python CLI/crobat_cli.py

# Override specific parameters
python CLI/crobat_cli.py --pair BTC-USD --duration 30

# Prompt for each parameter interactively
python CLI/crobat_cli.py --interactive

# Clear the output directory before recording
python CLI/crobat_cli.py --clear-runs
```

Available options:

--pair TEXT	Currency pair, e.g. XRP-USD
--duration INTEGER	Recording duration in seconds
--range INTEGER	Order book depth (number of price levels per side)
--sides TEXT	Comma-separated sides to record: bid,ask,signed
--filetype TEXT	Comma-separated output formats: csv,pkl,xlsx
--output-dir TEXT	Directory where output files are written [default: runs]
--clear-runs	Delete all existing files in the output directory before recording
--interactive	Prompt for each parameter (Enter to accept the shown default)

Output files are written to the runs/ directory by default, with UTC timestamps in the filename.

1.4 Using the Python API

You can integrate crobat directly into your own scripts:

```
from crobat import L2Recorder, SnapshotTimeoutError
from crobat.config import recording_defaults

class Settings:
    d = recording_defaults()
    currency_pair      = d['currency_pair']
    position_range     = d['position_range']
    recording_duration = d['recording_duration']
    sides              = d['sides']
    filetype           = d['filetype']
    output_dir         = 'runs'
```

(continues on next page)

(continued from previous page)

```
recorder = L2Recorder(Settings())
recorder.start()
```

After `start()` returns, the full session history is available on `recorder.book` — a *LimitOrderBook* instance:

```
# Most recent order book snapshot
recorder.book.latest_snapshot(side='signed')

# All bid-side events
recorder.book.bid_events

# All ask-side events
recorder.book.ask_events

# All signed events (includes market orders)
recorder.book.signed_events

# Most recent insertion, cancellation, or market order
recorder.book.last_inserted_order()
recorder.book.last_canceled_order()
recorder.book.last_market_order()

# Current mid-price and spread
recorder.book.mid_price
recorder.book.spread
```

1.5 Handling snapshot timeouts

If the WebSocket connection fails to deliver the initial snapshot, *SnapshotTimeoutError* is raised after all retry attempts are exhausted:

```
from crobat import L2Recorder, SnapshotTimeoutError

try:
    recorder = L2Recorder(Settings())
    recorder.start(snap_timeout=8.0, max_retries=2, retry_backoff=5.0)
except SnapshotTimeoutError as e:
    print(f"Failed to connect: {e}")
```

Note: On slow connections, increase `snap_timeout`. The default 8 seconds is sufficient for most conditions. At most `1 + max_retries` total connections are opened to avoid rate-limiting by Coinbase.

OUTPUT FORMAT

crobat produces time series files for each recording session. Files are named with a UTC timestamp suffix and written to the configured output directory (default: runs/).

2.1 Output files

Filename	Side	Description
L2_orderbook_volm_ask<timestamp>	ask	Volume snapshots, ask side
L2_orderbook_volm_bid<timestamp>	bid	Volume snapshots, bid side
L2_orderbook_volm_signed<timestamp>	both	Volume snapshots, signed order book
L2_orderbook_prices_ask<timestamp>	ask	Price snapshots, ask side
L2_orderbook_prices_bid<timestamp>	bid	Price snapshots, bid side
L2_orderbook_prices_signed<timestamp>	both	Price snapshots, signed order book
L2_orderbook_events_ask<timestamp>	ask	Event time series, ask side
L2_orderbook_events_bid<timestamp>	bid	Event time series, bid side
L2_orderbook_events_signed<timestamp>	both	Event time series, signed

2.2 Order book snapshots

Each snapshot row records the state of the order book at the moment an event was processed. Columns are ordinal positions from the best bid or best ask.

Single-sided snapshot (bid or ask):

An associated price snapshot is generated in the same format, with the price quote (e.g., USD per XRP) at each position.

2.3 Signed order book snapshot

The signed order book follows the convention from Cont, Kukanov and Stoikov (2011). Bid positions are negative, ask positions are positive. Position 0 is skipped — the best bid is -1, the best ask is 1.

Bid-side volume is stored as negative. An associated price snapshot is generated in the same format.

2.4 Event recordings

Events capture each limit order insertion (LO), cancellation (CO), and market order (MO) as they arrive from the exchange.

Single-sided events (bid or ask):

Timestamp	or- der_type	price_level	event_size	posi- tion	mid_price	spread
YYYY-MM-DD HH:MM:SS.ffff	LO/CO/MO	quote ccy	base ccy	ordinal	(ask+bid)/2 ask - bid	

Signed events add a `side` column and sign the event size according to the order flow convention: positive for buy-side activity (buy MO, sell CO, buy LO), negative for sell-side activity.

Note: The `Demo/` directory contains example output files from a 2020 recording session for reference.

API REFERENCE

3.1 `crobot.recorder`

The main entry point for live recording sessions. Defines *L2Recorder* and *SnapshotTimeoutError*.

exception `crobot.recorder.SnapshotTimeoutError`

Bases: `Exception`

Raised when the l2_data snapshot is not received within the allowed time.

class `crobot.recorder.L2Recorder(settings)`

Bases: `object`

Connects to the Coinbase Advanced Trade WebSocket feed and maintains a live Level 2 limit order book using the `coinbase-advanced-py` WSClient.

Subscribes to three channels for a given product:

- `level2` : snapshot + incremental order book updates
- `market_trades` : market order fills
- `ticker` : best bid / ask quotes, cached for spread calculations

session_start

UTC time at initialisation, used as the session start reference.

Type

`datetime`

book

The live limit order book state and history.

Type

LimitOrderBook

settings

Holds `currency_pair`, `position_range`, `recording_duration`, `sides`, `filetype`.

Type

`object`

snapshot_received

Guards against processing updates before the snapshot arrives.

Type

bool

decimal_places

Volume rounding precision. Set from the snapshot; defaults to 8.

Type

int

ws

The underlying coinbase-advanced-py websocket client.

Type

WSClient

on_open()

on_message(*raw*)

Route an incoming raw JSON string to the correct channel handler.

Parameters

raw (*str*) – Raw JSON string delivered by WSClient.

on_close()

start(*snap_timeout=8.0, max_retries=2, retry_backoff=5.0*)

Open the WebSocket, subscribe to all channels, wait for **recording_duration** seconds, then close cleanly.

The recording timer only starts once the l2_data snapshot is confirmed. If the snapshot is not received within **snap_timeout** seconds, the connection is closed and reopened after **retry_backoff** seconds.

Parameters

- **snap_timeout** (*float*) – Seconds to wait for the snapshot before treating the attempt as failed. Default 8s.
- **max_retries** (*int*) – Number of reconnection attempts after the first failure. Default 2.
- **retry_backoff** (*float*) – Seconds to wait between a failed attempt and the next reconnect. Default 5s.

Raises

SnapshotTimeoutError – If the snapshot is not received after all attempts are exhausted.

`crobat.recorder.main()`

3.2 crobat.orderbook

LimitOrderBook maintains all live order book state and history during a session. The module-level *apply_update()* function orchestrates each L2 update through the full mutation sequence.

class `crobat.orderbook.LimitOrderBook`

Bases: `object`

Maintains the state and full history of a Level 2 limit order book.

bid_history

Time series of bid-side snapshots. Each entry is `[datetime, snapshot]`.

Type

list of list

ask_history

Time series of ask-side snapshots. Each entry is `[datetime, snapshot]`.

Type

list of list

signed_history

Time series of signed order book snapshots (bids negative, asks positive).

Type

list of list

snapshot_bid

Current bid-side snapshot as `[[price, volume], ...]` best-to-worst.

Type

list of list

snapshot_ask

Current ask-side snapshot as `[[price, volume], ...]` best-to-worst.

Type

list of list

bid_events

Log of recorded bid-side order book changes.

Type

list of list

ask_events

Log of recorded ask-side order book changes.

Type

list of list

signed_events

Log of recorded signed order book changes.

Type

list of list

order_type

Type of the most recent event: 'insertion', 'cancellation', or 'market'. Written by [`apply\_update\(\)`](#) after each L2 update.

Type

str or None

token

True if the most recent update was recorded (within depth limit). Written by [`apply\_update\(\)`](#).

Type

bool

position

0-based snapshot index of the most recent change. Written by [`apply\_update\(\)`](#).

Type

int

event_size

Absolute size of the most recent order book change. Written by [`apply\_update\(\)`](#).

Type

float

min_dec

Number of decimal places used to round volumes. Set by [`initialise\_from\_snapshot\(\)`](#).

Type

int

Notes

`bid_range` and `ask_range` are read-only properties derived from the price column of `snapshot_bid` and `snapshot_ask` respectively. They are never stored separately — the snapshot is always the source of truth.

property bid_range

Prices of the current bid-side snapshot, best-to-worst (descending).

property ask_range

Prices of the current ask-side snapshot, best-to-worst (ascending).

`initialise_from_snapshot(msg, timestamp)`

Initialise the order book from a Level 2 snapshot message.

Populates `snapshot_bid`, `snapshot_ask`, and `min_dec` from the snapshot data. Appends the initial state to `bid_history`, `ask_history`, and `signed_history`.

Called once per session by `crobat.recorder.L2Recorder._handle_l2_data()` when the first snapshot event arrives.

Parameters

- **msg** (*dict*) – Snapshot dict with keys 'bids' and 'asks', each a list of [`price_str`, `quantity_str`] pairs ordered from best to worst.
- **timestamp** (*datetime*) – UTC timestamp at session start, used as the first history entry.

add_market_order(*event*, *event_log*)

Append a market order event to `event_log`, aggregating consecutive events at the same timestamp into a single entry.

If aggregation produces a net size of zero — which happens when a buy and sell of equal size arrive at the same timestamp — the combined entry is removed entirely. A zero-size market event carries no information about order flow direction.

Parameters

- **event** (*list*) – The new market order event to append.
- **event_log** (*list of list*) – The event log to append to (e.g. `signed_events`).

Returns

Updated event log.

Return type

list of list

remove_price_level(*snapshot*, *level_depth*, *match_index*, *result*)

Remove a price level from `snapshot` when its depth reaches zero.

Always runs after `update_level_depth()`. If the level was found and its new depth is zero, it is deleted and `result.recorded` is set to `True` with the removal position.

Parameters

- **snapshot** (*list of list*) –
- **level_depth** (*float*) –
- **match_index** (*int or None*) –
- **result** (*_UpdateResult*) – Result from `update_level_depth()`. Returned unchanged if `level_depth != 0` or `match_index` is `None`.

Returns

- *list of list* – Updated snapshot.
- *_UpdateResult* – Updated result.

update_level_depth(*snapshot, level_depth, match_index*)

Update the volume of an existing price level in *snapshot*.

Parameters

- **snapshot** (*list of list*) – The snapshot being edited.
- **level_depth** (*float*) – New quantity from the message.
- **match_index** (*int or None*) – Index of the matched price level, or None if not found.

Returns

- *list of list* – Updated snapshot.
- *_UpdateResult* – Carries *recorded*, *position*, *event_size*, *order_type*. *recorded* is False when the price was not found or the quantity was unchanged.

property mid_price

Best-bid/best-ask midpoint.

property spread

Best-ask minus best-bid.

trim_coordinator(*position, depth_limit*)

Return True if *position* is within *depth_limit* levels of the best price, False otherwise.

Parameters

- **position** (*int*) – Index where the change occurred.
- **depth_limit** (*int*) – Maximum recordable depth from the best price.

Return type

bool

append_snapshot_bid(*timestamp, price_level, depth_limit, result*)

Append the current bid snapshot and event to history.

Parameters

- **timestamp** (*datetime*) –
- **price_level** (*float*) –
- **depth_limit** (*int*) –
- **result** (*_UpdateResult*) – Carries *position*, *event_size*, *order_type* for the event.

append_snapshot_ask(*timestamp, price_level, depth_limit, result*)

Append the current ask snapshot and event to history.

Parameters

- **timestamp** (*datetime*) –

- **price_level** (*float*) –
- **depth_limit** (*int*) –
- **result** (*_UpdateResult*) – Carries position, event_size, order_type for the event.

append_signed_book(*timestamp, price_level, side, depth_limit, result*)

Append the current signed snapshot and event to history.

Parameters

- **timestamp** (*datetime*) –
- **price_level** (*float*) –
- **side** (*str*) – 'bid' or 'ask'
- **depth_limit** (*int*) –
- **result** (*_UpdateResult*) – Carries position, event_size, order_type for the event.

remove_market_cancel_duplicate(*event_log, order_type*)

Remove a duplicate event when a market order and a cancellation of the same size arrive back-to-back (the cancellation is the exchange's confirmation of the fill, not a separate order).

Checks the last two events in *event_log*. If their sizes match and the pair is a (market, cancellation) or (cancellation, market) sequence, the redundant entry is deleted in-place.

Parameters

- **event_log** (*list of list*) – The event log to check (e.g. *bid_events*).
- **order_type** (*str*) – The type of the event that just triggered this check.

last_inserted_order(*side='signed'*)

Return the most recent insertion event within the last 30 events.

Returns None if no insertion is found.

last_canceled_order(*side='signed'*)

Return the most recent cancellation event within the last 30 events.

Returns None if no cancellation is found.

last_market_order(*side='signed'*)

Return the most recent market order event within the last 30 events.

Returns None if no market order is found.

latest_snapshot(*side='signed'*)

Return the most recent order book snapshot.

Parameters

- **side** (*str*) – 'buy', 'sell', or 'signed'. Default 'signed'.

Returns

[datetime, snapshot]

Return type

list

last_market_depth(*side*, *depth_limit*='all')

Compute the total notional depth (price × volume) of the order book.

Parameters

- **side** (*str*) – 'buy' or 'sell'.
- **depth_limit** (int or 'all', optional) – Number of levels to include. Default is all levels.

Returns

Total notional depth.

Return type

float

Raises

ValueError – If side is not 'buy' or 'sell'.

crobat.orderbook.apply_update(*book*, *timestamp*, *side*, *price_level*, *level_depth*, *match_index*, *depth_limit*)

Apply a single L2 update to book for either the bid or ask side.

Runs the full mutation sequence with explicit data flow — each step receives the result of the previous one rather than reading back from shared instance state:

1. Update volume of an existing level ([*update_level_depth\(\)*](#))
2. Remove the level if volume is zero ([*remove_price_level\(\)*](#))
3. Insert a new level if the price was not found ([*_insert_price_level\(\)*](#))
4. Gate recording by depth ([*trim_coordinator\(\)*](#))
5. Append to history and check for market/cancel duplicates if within depth.

The *token*, *position*, *event_size*, and *order_type* attributes on *book* are written once at the end from the final result, so the public accessors (*last_inserted_order* etc.) always reflect the most recent update.

Parameters

- **book** ([*LimitOrderBook*](#)) –
- **timestamp** (*datetime*) –
- **side** (*str*) – 'bid' or 'ask'
- **price_level** (*float*) –
- **level_depth** (*float*) –

- **match_index** (*int* or *None*) – Index of the matched price level in the snapshot, or *None*.
- **depth_limit** (*int*) –

`crobat.orderbook.apply_bid_update(book, timestamp, price_level, level_depth, match_index, depth_limit)`

Alias for `apply_update()` with `side='bid'`.

`crobat.orderbook.apply_ask_update(book, timestamp, price_level, level_depth, match_index, depth_limit)`

Alias for `apply_update()` with `side='ask'`.

`crobat.orderbook.price_match(x, y)`

Return `True` if two prices are equal.

3.3 crobat.orderbook_helpers

Pure utility functions used internally by `crobat.orderbook`. No state, no side effects.

`crobat/orderbook_helpers.py`

Pure utility functions used by `crobat.orderbook.LimitOrderBook`. No state, no side effects — each function takes plain values and returns a result.

`crobat.orderbook_helpers.compute_sign(side, order_type)`

Return the sign (+1 or -1) for an order book event in the signed order book.

Follows the order flow convention from Cont, Kukanov and Stoikov (2011):

- **Ask side:** insertions and market orders are positive; cancellations are negative.
- **Bid side:** the sign is flipped relative to the ask side.

Parameters

- **side** (*str*) – Side of the order book where the event occurred. 'ask' or 'bid'.
- **order_type** (*str*) – Type of order book event. One of 'insertion', 'cancellation', or 'market'.

Returns

1 or -1.

Return type

`int`

`crobat.orderbook_helpers.compute_signed_position(position, side)`

Convert a zero-indexed ordinal position to a signed position.

The signed order book uses negative positions for the bid side and positive positions for the ask side, with position 0 skipped (best bid is -1, best ask is 1).

Parameters

- **position** (*int*) – Zero-indexed ordinal distance from the best bid or best ask.
- **side** (*str*) – Side of the order book. 'bid' or 'ask'.

Returns

Signed position: negative for bid, positive for ask.

Return type

int

`crobat.orderbook_helpers.compute_min_decimals(min_currency_denom, min_asset_value)`

Compute the number of decimal places needed to represent the smallest tradable quantity at the current price.

Uses the worst (deepest) bid price as a conservative floor. The result is used to round all volume values consistently throughout a session.

Parameters

- **min_currency_denom** (*float*) – Minimum currency denomination of the quote currency (e.g., 0.01 for one cent in USD).
- **min_asset_value** (*float*) – Lowest observed price of the base asset in the quote currency (e.g., the worst bid price in the order book snapshot).

Returns

Number of decimal places for the smallest tradable amount at the given price. Capped at 10 to avoid runaway precision on very cheap assets.

Return type

int

Raises

TypeError – If non-numeric values are passed. Ensure prices are cast to `float` before calling (e.g., `float(msg['price'])`).

3.4 crobat.filesave

Converts *LimitOrderBook* history arrays to DataFrames and writes output files. Called automatically by *on_close()* via *export_session()*.

`crobat/filesave.py`

Converts a completed *crobat.orderbook.LimitOrderBook* session object into pandas DataFrames and writes them to disk as CSV, PKL, or XLSX files.

Called automatically by *crobat.recorder.L2Recorder.on_close()* at the end of every recording session. Can also be called directly for post-hoc export of a history object.

`crobat.filesave.snapshot_history_to_records(history, pos_limit=5)`

Convert a single-sided order book history into two lists of dicts suitable for DataFrame construction.

Each entry in `history` is a `[timestamp, snapshot]` pair where `snapshot` is a list of `[price, volume]` pairs ordered best-to-worst.

Parameters

- **history** (*list of list*) – Use `bid_history` or `ask_history` from a `crobat.orderbook.LimitOrderBook` instance.
- **pos_limit** (*int, optional*) – Number of price levels to include per snapshot. Default 5.

Returns

- **volume_records** (*list of dict*) – Keys: `'time', '1' ... str(pos_limit)`.
- **price_records** (*list of dict*) – Same structure, values are prices instead of volumes.

`crobat.filesave.signed_history_to_records(history, pos_limit=5)`

Convert a signed order book history into two lists of dicts suitable for DataFrame construction.

Bid levels carry negative positions and negated volumes; ask levels carry positive positions. Follows Cont, Kukanov and Stoikov (2011).

Parameters

- **history** (*list of list*) – Use `signed_history` from a `crobat.orderbook.LimitOrderBook` instance.
- **pos_limit** (*int, optional*) – Number of price levels per side. Default 5.

Returns

- **volume_records** (*list of dict*)
- **price_records** (*list of dict*)

`crobat.filesave.events_to_records(events, signed=False)`

Convert a raw event log into a list of dicts with human-readable column names, suitable for DataFrame construction.

Parameters

- **events** (*list of list*) – Each element is either:
 - `[time, order_type, price, size, position, mid_price, spread]` for bid/ask events, or
 - `[time, order_type, price, size, position, side, mid_price, spread]` for signed events.
- **signed** (*bool, optional*) – True for signed event logs, which carry an extra side field.

Return type

list of dict

`crobat.filesave.clear_output_dir(output_dir)`

Delete all files in `output_dir`, leaving the directory itself intact.

Only regular files are removed. Subdirectories are left untouched. If the directory does not exist, this is a no-op.

Parameters

output_dir (*str*) –

Returns

Number of files deleted.

Return type

int

`crobat.filesave.save_pickle(title, records, output_dir='.', drop_init_row=True)`

Save a list of dicts to a `.pkl` file.

`crobat.filesave.save_csv(title, records, output_dir='.', drop_init_row=True)`

Save a list of dicts to a `.csv` file.

`crobat.filesave.save_excel(title, records, output_dir='.', drop_init_row=True)`

Save a list of dicts to a `.xlsx` file.

`crobat.filesave.export_session(book, position_range, sides, filetype, include_events=True, output_dir='.')`

Export a completed session to disk.

Writes one volume file, one price file, and (optionally) one events file per requested side, in each requested format.

Called automatically by `crobat.recorder.L2Recorder.on_close()`.

Parameters

- **book** (`crobat.orderbook.LimitOrderBook`) –
- **position_range** (*int*) – Number of price levels per side per snapshot row.
- **sides** (*list of str*) – Any combination of 'bid', 'ask', 'signed'.
- **filetype** (*list of str*) – Any combination of 'csv', 'pkl', 'xlsx'.
- **include_events** (*bool, optional*) – Write event log files in addition to snapshot files. Default True.
- **output_dir** (*str, optional*) – Directory to write into. Created if absent. Default '.'.

3.5 crobat.config

Loads Coinbase credentials and recording defaults from `config.ini` and `cdp_api_key.json`.

`crobat/config.py`

Loads Coinbase API credentials and recording defaults for a session.

Credentials are read from `cdp_api_key.json` in the project root (downloaded from the Coinbase Developer Portal). If the file is absent, `coinbase_credentials()` returns an empty dict and the `coinbase-advanced-py` WSClient falls back to the `COINBASE_API_KEY` / `COINBASE_API_SECRET` environment variables.

Recording defaults are read from the `[recording]` section of `config.ini` in the project root.

Usage:

```
from crobat.config import coinbase_credentials, recording_defaults

creds      = coinbase_credentials()    # passed directly to WSClient
defaults = recording_defaults()        # dict of session parameters
```

`crobat.config.coinbase_credentials()` → dict

Return the keyword arguments needed to authenticate a WSClient.

Looks for `cdp_api_key.json` in the project root. If found, returns `{'key_file': <path>}`. If not found, returns `{}` so that WSClient falls back to the `COINBASE_API_KEY` and `COINBASE_API_SECRET` environment variables.

Returns

Either `{'key_file': str}` or `{}`.

Return type

dict

`crobat.config.recording_defaults()` → dict

Return recording session defaults from `config.ini`.

Reads the `[recording]` section. If the section is absent, returns an empty dict and callers should supply their own values.

Returns

Keys and their fallback values if not set in `config.ini`:

- `currency_pair` (str): 'XRP-USD'
- `position_range` (int): 5
- `recording_duration` (int): 10
- `sides` (list of str): ['bid', 'ask', 'signed']
- `filetype` (list of str): ['csv']

Return type
dict

REFERENCES

crobot's event recording conventions follow the market microstructure literature. The output formats for order book snapshots, event time series, and signed order flow are designed to be compatible with the models described in the papers below.

4.1 Order book dynamics and queue-reactive models

Huang W., Lehalle C.A. and Rosenbaum M. (2015). *Simulating and analyzing order book data: The queue-reactive model*. Journal of the American Statistical Association. <https://arxiv.org/pdf/1312.0563.pdf>

4.2 Stochastic order book models

Cont R., Stoikov S. and Talreja R. (2010). *A stochastic model for order book dynamics*. Operations Research. <https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.139.1085&rep=rep1&type=pdf>

4.3 Signed order flow and price impact

Cont R., Kukanov A. and Stoikov S. (2011). *The price impact of order book events*. Journal of Financial Econometrics. <https://arxiv.org/pdf/1011.6402.pdf>

This paper defines the signed order book convention used in crobat: positive order flow from buy MOs, sell COs, and buy LOs; negative from sell MOs, buy COs, and sell LOs.

4.4 Spoofing and manipulation

Cartea A., Jaimungal S. and Wang Y. (2020). *Spoofing and Price Manipulation in Order Driven Markets*. Applied Mathematical Finance. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3431139

4.5 Cryptocurrency order flow analysis

Silyantev E. (2019). *Order flow analysis of cryptocurrency markets*. Digital Finance. <https://link.springer.com/article/10.1007/s42521-019-00007-w>

This paper demonstrates Order Flow Imbalance (OFI) and Trade Flow Imbalance applied to BTC-USD, providing a working model for the type of analysis crobat's output is designed to support.

4.6 Related work

This library grew out of research presented in:

Perez I.E. (2021). *A Study of CUSUM Statistics on Bitcoin Transactions*. CUNY Academic Works. https://academicworks.cuny.edu/cgi/viewcontent.cgi?article=1682&context=hc_sas_etds

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

C

`crobat.config`, [21](#)
`crobat.filesave`, [18](#)
`crobat.orderbook`, [11](#)
`crobat.orderbook_helpers`, [17](#)
`crobat.recorder`, [9](#)

INDEX

A

`add_market_order()` (*crobat.orderbook.LimitOrderBook* method), 13

`append_signed_book()` (*crobat.orderbook.LimitOrderBook* method), 15

`append_snapshot_ask()` (*crobat.orderbook.LimitOrderBook* method), 14

`append_snapshot_bid()` (*crobat.orderbook.LimitOrderBook* method), 14

`apply_ask_update()` (*in module crobat.orderbook*), 17

`apply_bid_update()` (*in module crobat.orderbook*), 17

`apply_update()` (*in module crobat.orderbook*), 16

`ask_events` (*crobat.orderbook.LimitOrderBook* attribute), 11

`ask_history` (*crobat.orderbook.LimitOrderBook* attribute), 11

`ask_range` (*crobat.orderbook.LimitOrderBook* property), 12

B

`bid_events` (*crobat.orderbook.LimitOrderBook* attribute), 11

`bid_history` (*crobat.orderbook.LimitOrderBook* attribute), 11

`bid_range` (*crobat.orderbook.LimitOrderBook* property), 12

`book` (*crobat.recorder.L2Recorder* attribute), 9

C

`clear_output_dir()` (*in module crobat.filesave*), 19

`coinbase_credentials()` (*in module crobat.config*), 21

`compute_min_decimals()` (*in module crobat.orderbook_helpers*), 18

`compute_sign()` (*in module crobat.orderbook_helpers*), 17

`compute_signed_position()` (*in module crobat.orderbook_helpers*), 17

`crobat.config` module, 21

`crobat.filesave` module, 18

`crobat.orderbook` module, 11

`crobat.orderbook_helpers` module, 17

`crobat.recorder` module, 9

D

`decimal_places` (*crobat.recorder.L2Recorder* attribute), 10

E

`event_size` (*crobat.orderbook.LimitOrderBook* attribute), 12

`events_to_records()` (*in module crobat.filesave*), 19

`export_session()` (*in module crobat.filesave*), 20

I

`initialise_from_snapshot()` (*crobat.orderbook.LimitOrderBook* method), 12

L

`L2Recorder` (*class in crobat.recorder*), 9

`last_canceled_order()` (*crobat.orderbook.LimitOrderBook* method), 15

`last_inserted_order()` (*crobat.orderbook.LimitOrderBook* method), 15

`last_market_depth()` (*crobat.orderbook.LimitOrderBook* method), 16

`last_market_order()` (*crobat.orderbook.LimitOrderBook* method), 15

`latest_snapshot()` (*crobat.orderbook.LimitOrderBook* method), 15

`LimitOrderBook` (class in *crobat.orderbook*), 11

M

`main()` (in module *crobat.recorder*), 10

`mid_price` (*crobat.orderbook.LimitOrderBook* property), 14

`min_dec` (*crobat.orderbook.LimitOrderBook* attribute), 12

module

- `crobat.config`, 21
- `crobat.filesave`, 18
- `crobat.orderbook`, 11
- `crobat.orderbook_helpers`, 17
- `crobat.recorder`, 9

O

`on_close()` (*crobat.recorder.L2Recorder* method), 10

`on_message()` (*crobat.recorder.L2Recorder* method), 10

`on_open()` (*crobat.recorder.L2Recorder* method), 10

`order_type` (*crobat.orderbook.LimitOrderBook* attribute), 12

P

`position` (*crobat.orderbook.LimitOrderBook* attribute), 12

`price_match()` (in module *crobat.orderbook*), 17

R

`recording_defaults()` (in module *crobat.config*), 21

`remove_market_cancel_duplicate()` (*crobat.orderbook.LimitOrderBook* method), 15

`remove_price_level()` (*crobat.orderbook.LimitOrderBook* method), 13

S

`save_csv()` (in module *crobat.filesave*), 20

`save_excel()` (in module *crobat.filesave*), 20

`save_pickle()` (in module *crobat.filesave*), 20

`session_start` (*crobat.recorder.L2Recorder* attribute), 9

`settings` (*crobat.recorder.L2Recorder* attribute), 9

`signed_events` (*crobat.orderbook.LimitOrderBook* attribute), 11

`signed_history` (*crobat.orderbook.LimitOrderBook* attribute), 11

`signed_history_to_records()` (in module *crobat.filesave*), 19

`snapshot_ask` (*crobat.orderbook.LimitOrderBook* attribute), 11

`snapshot_bid` (*crobat.orderbook.LimitOrderBook* attribute), 11

`snapshot_history_to_records()` (in module *crobat.filesave*), 18

`snapshot_received` (*crobat.recorder.L2Recorder* attribute), 9

`SnapshotTimeoutError`, 9

`spread` (*crobat.orderbook.LimitOrderBook* property), 14

`start()` (*crobat.recorder.L2Recorder* method), 10

T

`token` (*crobat.orderbook.LimitOrderBook* attribute), 12

`trim_coordinator()` (*crobat.orderbook.LimitOrderBook* method), 14

U

`update_level_depth()` (*crobat.orderbook.LimitOrderBook* method), 13

W

`ws` (*crobat.recorder.L2Recorder* attribute), 10